

# 第 48 讲实验设计：CNN 基础与 Conv2d Shape 验证

## 实验总目标

本实验不是为了训练大型视觉模型，而是用最小可控例子验证 CNN 的 8 个底层命题：局部乘加、Kernel 滑动、框架约定、Shape 公式、参数共享、多 Channel、Pooling、平移等变性，以及一个最小 CNN 的完整 Shape Flow。

### 总假设

如果我们真正理解 CNN，那么应该能够在运行代码之前预测：输出 Shape、参数量、局部响应、Pooling 结果，以及输入局部平移后 Feature Map 的变化方向。

## 1 实验 1：手写二维卷积与 PyTorch 对照

**研究问题：**一个 Feature Map 数字到底怎样产生？

**设计：**构造  $5 \times 5$  二值输入和  $3 \times 3$  边缘 Kernel，用 NumPy 双循环逐位置计算，再与 `F.conv2d` 对照。

**验收：**手写输出应为：

$$\begin{bmatrix} 0 & 3 & 3 \\ 0 & 3 & 3 \\ 0 & 3 & 3 \end{bmatrix}$$

且与 PyTorch 最大误差为 0。

## 2 实验 2：cross-correlation 与严格 convolution

**研究问题：**PyTorch 是否会翻转 Kernel？

**设计：**使用非对称 Kernel，分别计算“不翻转”和“翻转后”的结果。

**验收：**不翻转版本与 PyTorch 一致；严格翻转版本应出现明显差异。

## 3 实验 3：Stride / Padding 与输出 Shape

**研究问题：**输出空间大小是否符合公式？

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

分别测试多组 Kernel / Stride / Padding，并与 PyTorch Shape 对照。

## 4 实验 4: Parameter Sharing 与参数量

**研究问题：**为什么卷积参数量远少于巨大 Dense Mapping?

**设计：**Conv2d(3,16,3,padding=1) 输入 32×32 RGB; 计算卷积参数量并与假设的 3072→16384 Dense 层比较。

**实际结果：**

$$Conv = 448, \quad Dense = 50,348,032$$

差约 112,384 倍。

## 5 实验 5: Multi-Channel Conv2d

**研究问题：**一个输出 Channel 是否会汇总所有输入 Channels?

手写：

$$X : (2, 4, 4), \quad W : (2, 2, 3, 3), \quad b : (2, )$$

输出应为：

$$Y : (2, 2, 2)$$

并与 PyTorch 逐元素一致。

## 6 实验 6: Max Pooling

**研究问题:**  $2 \times 2$  Max Pool 怎样把  $4 \times 4$  压成  $2 \times 2$ ?

输入:

$$\begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 6 & 1 & 2 \\ 0 & 2 & 8 & 7 \\ 1 & 5 & 3 & 9 \end{bmatrix}$$

预期:

$$\begin{bmatrix} 6 & 2 \\ 5 & 9 \end{bmatrix}$$

## 7 实验 7: Translation Equivariance

**研究问题:** 局部模式右移 1 格后, 卷积响应是否也右移?

**设计:** 在  $9 \times 9$  输入中放一个单位脉冲, 再向右移动 1 格, 使用同一  $3 \times 3$  Kernel。

**验收:** 最大响应位置应从 (4, 4) 移到 (4, 5); 忽略边界后平移后的 Feature Map 应重合。

## 8 实验 8: TinyCNN Shape Flow

构建:

$$Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Linear$$

输入:

$$(5, 1, 8, 8)$$

预期逐层:

$$(5, 4, 8, 8) \rightarrow (5, 4, 8, 8) \rightarrow (5, 4, 4, 4) \rightarrow (5, 64) \rightarrow (5, 2)$$

总参数量应为 170。

## 实验变量与控制

- 所有输入都使用固定小矩阵, 保证可手算与复核。
- PyTorch 与 NumPy 使用相同 Weight / Bias, 不让随机初始化成为 confounder。
- Translation 实验把模式放在远离边界的位置, 避免 Padding 边界效应混入主要结论。
- 本实验不比较视觉 SOTA, 不把“某个手写 Kernel”解释成真实 CNN 学到的唯一机制。

## 科研式记录模板

运行代码后至少记录: PyTorch 版本、随机种子、Input Shape、Weight Shape、Stride、Padding、参数量、预期 Shape、实际 Shape、最大数值误差。

### 【常见误区 / 风险】

不要只截图最后的 Feature Map。真正可复现的实验必须保留: 输入、Kernel / Weight、Shape、超参数和代码。否则无法判断差异来自 Kernel、Padding、Channel 顺序还是框架设置。